React Micro-Frontend Assignment

Position: React Developer – Frontend Expert (Micro-Frontend)

Duration: 1 Day

Objective

Build a Banking Micro-Frontend Application consisting of:

```
App Shell
 ├── Loan MFE
 └── Onboarding MFE
```

The architecture must include:
- pnpm workspace monorepo
- Vite Module Federation
- Shared UI component library
- Global state sharing
- Lazy loaded micro-frontends

Monorepo Structure

```
apps
 ├── app-shell
 ├── loan-mfe
 └── onboarding-mfe

packages
 ├── ui-library
 └── store
```

pnpm Workspace Configuration

```
packages:
  - apps/*
  - packages/*
```

Mandatory Requirements

The implementation must include:
- pnpm workspace
- Vite Module Federation
- Shared UI component library
- Global state management
- Lazy loading of micro frontends
- TailwindCSS based design system
- Wizard based onboarding flow
- Camera based dummy liveness verification

Shared UI Library

Location: packages/ui-library

Create reusable components:
- Button
- Input
- Card
- Stepper
- FileUploader
- CameraPreview
- Loader

All applications must use components from this library.

Global State

Location: packages/store

Use Zustand or React Context.

Example global state:
- userProfile
- onboardingProgress
- loanApplication

State should be shared across all micro frontends.

Lazy Loading

Micro-frontends must be lazy loaded in the app shell using React.lazy and Suspense.
Example:

```
const LoanApp = React.lazy(() => import("loan-mfe/App"));
const OnboardingApp = React.lazy(() => import("onboarding-mfe/App"));
```

App Shell (Host Application)

Responsibilities:
- Navigation
- Layout
- Route management
- Loading micro-frontends

Routes:
/ → Dashboard
/loan → Loan MFE
/onboarding → Onboarding MFE

Use TailwindCSS for layout and styling.

Onboarding Micro-Frontend

Route: /onboarding

The onboarding process must be wizard based:

Step 1 → Personal Information
Step 2 → Address Information
Step 3 → Income Information
Step 4 → Review & Submit

Step 1 – Personal Information

Fields:
- Full Name
- Date of Birth
- Phone
- Email

Additional Features:
- NID Upload (Front & Back)
- Face Liveness Verification

NID Upload should use the shared FileUploader component.

Face Liveness (Dummy):
- Open camera using browser API
- Capture image
- Simulate verification result after 2–3 seconds

Step 2 – Address Information

Fields:
- Present Address
- City
- District
- Postal Code

Step 3 – Income Information

Fields:
- Occupation
- Monthly Income
- Company Name

Step 4 – Review & Submit

Display all collected information including:
- Personal Info
- Address
- Income

- NID Upload Status
- Liveness Verification Status

User can submit onboarding after review.

Loan Micro-Frontend

Route: /loan

Loan application must be multi-step:

Step 1 → Loan Product Selection
Step 2 → Personal Information
Step 3 → Summary & Submit

Step 1 – Loan Product Selection

Fetch product data from:
Create a JSON file and pick info from the JSON like as API

Display loan cards with:
- Loan Name
- Interest Rate
- Maximum Amount
- Select Button

Step 2 – Personal Information

Fields:
- Full Name
- Phone
- Email
- Monthly Income

Optional:
- Loan Amount
- Loan Tenure

Step 3 – Summary

Display summary:
- Selected Loan
- Loan Amount
- Interest Rate
- Applicant Information

User submits loan application and sees success message.

UI Design Rules

Use TailwindCSS.

Color System:
Primary → blue-600
Background → gray-100
Card → white
Success → green-500
Error → red-500

Card Style:
rounded-xl
shadow
p-6

Spacing:
gap-4
p-4
p-6

Module Federation

Each micro frontend must expose its App component using Vite Module Federation.

Example:
```
federation({
 name: "loan-mfe",
 filename: "remoteEntry.js",
```

```
 exposes: {
   "./App": "./src/App"
 }
})
```

Development Ports

app-shell → localhost:3000
loan-mfe → localhost:3001
onboarding-mfe → localhost:3002

Expected Folder Structure

```
apps
 ├── app-shell
 ├── loan-mfe
 └── onboarding-mfe

packages
 ├── ui-library
 └── store
```

Submission

Submit a GitHub repository.

README must include:
pnpm install
pnpm dev